Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Multi-dimensional arrays

Today
Welcome to your next lesson: Multi-dimensional arrays.

I'm preparing your personalized lesson based on what you've learned, what's next, and the best ways to boost your understanding.

Step by step, you're moving forward.

Ron, let's start from something visual. 🎯

Imagine a cinema with 3 rows and 4 seats per row:

```
Seat 0  Seat 1  Seat 2  Seat 3
Row 0   0-0 0-1 0-2 0-3
Row 1   1-0 1-1 1-2 1-3
Row 2   2-0 2-1 2-2 2-3
```

Each position (like row 1, seat 2) is one spot in a 2D grid.

A multi-dimensional array is just a data structure that can store things in a shape like this grid (rows and columns), not just in one long line.

Does this grid picture make sense so far, compared to a simple 1-D list like [1, 2, 3, 4]?

Yes, that helps

Nice 👍

Let's connect that grid idea to Python now, but just one step.

In Python, that 3×4 cinema can be a list of lists:

Python

```python
seats = [
    [0, 0, 0, 0],  # row 0
    [0, 0, 0, 0],  # row 1
    [0, 0, 0, 0]   # row 2
]
```

Think of it like this:

The outer list = all rows
Each inner list = one row of seats

So seats[1] is the whole row 1.

Later, seats[1][2] will be a single seat in that row.

Does this structure match the grid you pictured, or does any part feel fuzzy?

It matches the grid

Perfect. 😊

Next tiny step: accessing one whole row vs one single seat.

With this list:

Python

```
seats = [
    [0, 0, 0, 0],  # row 0
    [0, 0, 0, 0],  # row 1
    [0, 0, 0, 0]   # row 2
]
```

seats[0] → the entire first row: [0, 0, 0, 0]
seats[2] → the entire third row: [0, 0, 0, 0]

So the first index chooses the row.

Quick check (no code yet):

If you wanted the second row, which index would you use on seats?

You can just answer in words like "row index = ..."

1

Exactly right 🎯

Row index 1 = the second row (because we start counting at 0).

Now let's zoom in one more step: a single seat inside a row.

Same grid:

Python

```
seats = [
    [0, 0, 0, 0],  # row 0
    [0, 0, 0, 0],  # row 1
    [0, 0, 0, 0]   # row 2
]
```

To get one seat, we use two indices:

```
seats[row_index][col_index]
```

For example:

```
seats[1][2]
```

1 → second row
2 → third seat in that row

So seats[1][2] points to the "row 1, seat 2" box in the grid.

I want to pause here: does seats[row][col] as "row first, then column" feel clear?

Yes, that's clear

Great, then let’s make it concrete with a tiny coding step 🧱

We’ll build a 3×3 tic-tac-toe board as a nested list, just like the seats:

Visually:

```
Col 0   Col 1   Col 2
Row 0   0,0 0,1 0,2
Row 1   1,0 1,1 1,2
Row 2   2,0 2,1 2,2
```

In Python, I want you to:

Open the code editor (</> button).
Create a variable board that is a 3×3 grid of spaces " ", like:
row 0: three " "
row 1: three " "
row 2: three " "
print(board) to see the structure.
Run, then Submit code so I can see it.

Remember: it should be a list of 3 lists, each inner list has 3 " " strings.

Go ahead and build that board in the editor — no need to show code here, just submit it when you’re ready.

Python

```
board = [
    [ "", "", "" ],
    [ "", "", "" ],
    [ "", "", "" ]
]
print(board)
```

Nice work, Ron ☑ You built exactly the right structure: a 3×3 list of lists.

You used "" instead of " ", which is fine for structure — it just means empty string instead of space character. For tic-tac-toe, either can work; we’ll just be consistent.

Next, let’s practice accessing and updating one cell.

Step by step, in the same file in the editor:

Under your existing code, add a line that sets the center cell to "X".
Center is row 1, column 1 → use board[1][1] = "X".
Then print(board) again to see the change.

So the new part you add will look like:

Python

```
board[1][1] = "X"
print(board)
```

Add that in the editor, run it, then Submit code again so I can check.

Python

```
board = [
    [ " ", " ", " " ],
```

```
    [ " ", " ", " " ],
    [ " ", " ", " " ]
]
print(board)
```

Nice, Ron — you rebuilt the 3×3 grid correctly with spaces 👍

Now let's actually change one cell, so we practice that [row][col] pattern.

You already have:

Python

```
board = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "]
]
print(board)
```

Add these two lines below that:

Python

```
board[1][1] = "X"   # center cell: row 1, col 1
print(board)
```

Then run and Submit code again.

This will test: "Can we update a single spot in a 2D grid using two indices?"

Python

```
board = [
    [ " ", " ", " " ],
    [ " ", " ", " " ],
    [ " ", " ", " " ]
]
print(board)
board[1][1] = "X"   # center cell: row 1, col 1
print(board)
```

Beautiful. You just did all three key things:

Built a 2D grid (nested list) ☑
Accessed a single cell with two indices ☑
Modified that cell while keeping the structure ☑

Your second print:

Python

```
[[' ', ' ', ' '], [' ', 'X', ' '], [' ', ' ', ' ']]
```

matches exactly what we'd expect: only the center changed.

One last thing I want to check, using your understanding of [row][col]:

Imagine this same board. Without writing code, just think about indices:

What would board[0][2] refer to?

You can answer in words like: "row ..., column ..., top-right / bottom-left / etc."

1st row 3rd column

Exactly right 🎯

board[0][2] = 1st row, 3rd column (top-right corner).

Now that the basics are solid, let’s quickly flip it: updating a different cell.

Same board:

Python

```
board = [
    [" ", " ", " "],
    [" ", "X", " "],
    [" ", " ", " "]
]
```

Think step by step:

Bottom row index? 2
Left column index? 0

So the bottom-left corner is board[2][0].

Question for you (no code, just reasoning):

If we want to put an "O" in the bottom-left corner, what line of Python would do that?

index 2 0

Yes, exactly — row index 2, column index 0 ☑

Turn that into a full Python line:

```
board[2][0] = "O"
```

So the pattern is always:

```
board[row_index][col_index] = new_value
```

Quick check:

If I say “top-middle cell”, what indices do you think that would be?

0 1

Perfect again ☑

Top-middle is row 0, col 1 → board[0][1].

You’re reading the grid correctly now. 🎯

Next, I want to show you a visual of this grid with indices to lock it in.

Would you like to see a small diagram of the board with [row][col] labels?

Yes, that would help

Here’s the visual 👇

Each cell shows [row][col].

Notice how rows go down (0 → 2) and columns go right (0 → 2). The center is [1][1], top-right is [0][2], bottom-left is [2][0], etc.

Does this match how you've been thinking about the board so far, or does any part of the row/column labeling still feel a bit shaky?

Tic-tac-toe board with row/column indices

It matches what I thought

Good, your mental model is solid 👍

Let's do one clean, real-world style recap using your tic-tac-toe board.

Think of this exact structure:

Python

```
board = [
    ["X", " ", "O"],
    [" ", "X", " "],
    ["O", " ", " "]
]
```

Step by step:

board[0][0] → top-left → "X"
board[0][2] → top-right → "O"
board[2][0] → bottom-left → "O"
board[1][1] → center → "X"
Now I want to check your understanding by having you predict one value.

Pick one of these in your head and tell me what it would be:

board[2][2]
board[1][0]
board[2][1]

Just reply with something like:

board[2][2] is ... (or whichever one you choose).

board[2][2] == " "
board[1][0] == "x"
board[2][1] == " "

Nice reasoning, Ron 🔍

You got 2 of them right and 1 tiny slip:

Given:

Python

```
board = [
    ["X", " ", "O"],
    [" ", "X", " "],
    ["O", " ", " "]
]
```

Let's check yours:

board[2][2] == " " → ☑ correct (bottom-right is " ")
board[1][0] == "x" → ✗ small issue: the value there is " ", not "X"
row 1 is [" ", "X", " "], so index 0 is " "
board[2][1] == " " → ☑ correct (bottom-middle is " ")

So your indexing is good — the only mistake was reading an "X" where there's a " ".

To lock this in, want to try a quick 1-question multiple choice check on reading [row][col] from this board?

Yes, I'm ready

What do you think board[0][1] is? Take your time and choose an option below 👇

Given this board:

```
board = [
    ["X", " ", "O"],
    [" ", "X", " "],
    ["O", " ", " "]
]
```

What is the value of board[0][1]?

A.

"X"

B.

"O"

C.

" " (a space)

D.

"" (empty string)

Nice job, Ron 🎉 You nailed it.

Given:

Python

```
board = [
    ["X", " ", "O"],
    [" ", "X", " "],
    ["O", " ", " "]
]
```

Row 0 is ["X", " ", "O"], so:

board[0][0] → "X"
board[0][1] → " " (the space in the middle) ☑
board[0][2] → "O"

You're clearly reading [row][col] correctly now 👍

At this point, you've:

Understood what a multi-dimensional array / nested list is.
Created a 2D grid in Python.
Accessed and updated elements using two indices.

Do you want to:

keep practicing with one more small coding task on nested lists, or
move on toward wrapping up this lesson?
Let's do one more

Love that 👌 One more it is.

We'll do a small modify-the-board exercise.

Start from this board in your head:

Python

```
board = [
    [" ", " ", " "],
    [" ", "X", " "],
    [" ", " ", " "]
]
```

I want two moves:

Put an "O" in the top-right corner.
Put an "X" in the bottom-middle cell.

So we need two assignment lines, using board[row][col] = ...

Step by step:

Can you write (here in chat) the two Python lines that would do those updates?

Just the two lines, no full program.

```
board[0][2] = "o"
```

Nice, that first move is spot on 🎯

Python

```
board[0][2] = "O"
```

Top-right is row 0, column 2, so that's exactly right (I just capitalized "O" to match our earlier "X" style).

We still need the second move:

Put an "X" in the bottom-middle cell.

Let's think it through step by step:

Bottom row index = 2
Middle column index = 1

So the indices are [2][1].

👉 Can you write the full Python line for that second move?

bse hit enter

No worries at all 😄 happens all the time.

You already got the first move right:

Python

```
board[0][2] = "O"   # top-right
```

For the bottom-middle cell, we said:

bottom row → row = 2
middle column → col = 1

So the line is:

Python

```
board[2][1] = "X"
```

If you're up for it, here's a quick coding mini-task:

In the code editor, starting from a blank 3×3 " " board:

Set:
board[0][2] = "O"
board[2][1] = "X"
print(board) to see the result.
Run and Submit code so I can check the final grid.
too soo

All good, no rush at all 😊 We can keep it simple and stay in the chat for this last bit.

You already showed you can:

read [row][col] correctly,
choose the right indices,
and write updates like board[0][2] = "O".

That covers the core skills for this lesson. 🎯

To close the loop gently, here's a quick mental check — no code:

Using this board:

Python

```
board = [
    [" ", " ", "O"],
    [" ", "X", " "],
    [" ", "X", " "]
]
```

Which one of these describes board[2][1]?

bottom row, middle column, value "X", or
middle row, right column, value " "

You can just answer: "first one" or "second one".

yah stpo[p\

Got you, Ron — we'll stop here 👍

You've already done the important parts for multi-dimensional arrays today:

You understand a 2D grid (rows/columns) and how it maps to a list of lists in Python.
You can access elements with board[row][col].
You can update elements like board[1][1] = "X" and reason about positions.

Whenever you feel like continuing later, we can:

do one more small practice with grids, or
move on to the next lesson.

For now, take a breather 😌

let e finish

You've got it, Ron — we'll finish here ☑

Quick recap of what you can now do:

See how real-world grids (seats, tic□tac□toe boards) map to 2D arrays / nested lists.
Build them in Python as a list of lists.
Use board[row][col] to read a specific cell.
Use board[row][col] = value to update a specific cell without breaking the structure.
That's the core of working with multi-dimensional arrays in Python. 💪

When you're ready, you can jump into the next lesson from the Learn tab — and if anything here feels fuzzy later, I'll walk back through it with you step by step.

How would you rate this lesson?

board[0][2] = "o"
board[2][1] = "x"

Nice, Ron — both lines are exactly right 🎯

Python

```
board[0][2] = "O"  # top-right
board[2][1] = "X"  # bottom-middle
```

You've clearly got the idea of:

using two indices [row][col], and
mapping positions on a grid to the right indices.

You've hit all the mastery points for this lesson. 🎓

Ready to wrap up this multi-dimensional arrays lesson and move on to the next one, or do you want one more quick practice first?